ECEN 248 - Lab Report

Lab Number: 11

Lab Title: The Traffic Light Controller

Section Number: 509

Student's Name: Saira Khan

Date: 12/7/2022

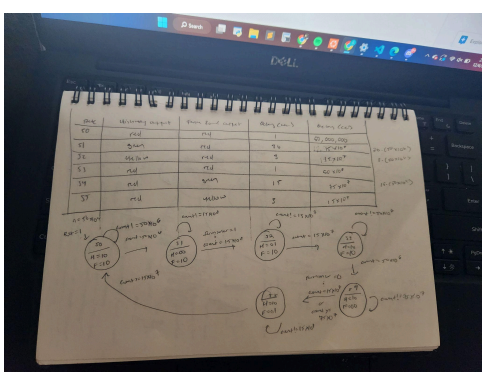
TA: Sri Kodi

Objectives:

The purpose of this lab is to continue working with finite-state machines. This lab, however, focuses on the Mealy machine in order to implement a traffic light controller. The Mealy machine depends on both the state of the machine as well as the input, unlike the Moore machine which only depends on the state of the machine. This lab works with implementing pauses between state transitions too. In order to do this, counters will be used.

Design:

In order to complete this lab, a state diagram of the traffic controller light FSM is created. From that state diagram, a behavioral Verilog file is created to describe the FSM. Next, a file called tlc_controller is created using the provided code, and the blanks are filled in to instantiate the finite state machine and a counter with a synchronous reset is described. Two source files are copied into the lab 11 directory, synchronizer.v and tlc_controller.xdc, and added to the design along with the source file. The design is then synthesized and implemented. The design is loaded onto the FPGA board in order to test its functionality. The BTN0 button is pressed in order to observe the output of the LEDs, where LED3 and LED2 are for the highway signal and LED1 and LED0 are for the farm signal. For experiment two, the design is modified to support a new sensor, the farm sensor, which asserts the signal when a car is on the farm road in either direction. An input port, farmsensor, is added to the FSM and the FSM is modified. The highway light is set to remain green as long as the sensor signal is low and must be green for at least 30 seconds at a time. Once the light has transitioned to green, the farm road light will remain green for three seconds. When the sensor signal is high, the farm road light will remain green for less than 15 seconds, and the lights will transition from green to yellow to red. The tlc_controller.v file and tlc_controller.xdc file are modified to accommodate the new sensor. BTN1 on the ZYBO Z7-10 board is used to mimic the sensor output. The design is then loaded onto the board by re-synthesizing and implementing it again.



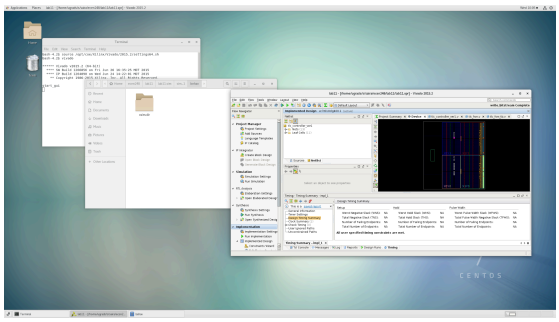
Results:

The lab went as planned, as the traffic controller was implemented and synthesized on the board correctly. A farm sensor was then incorporated into the board, implemented, and then synthesized once again to make sure the code was running correctly.

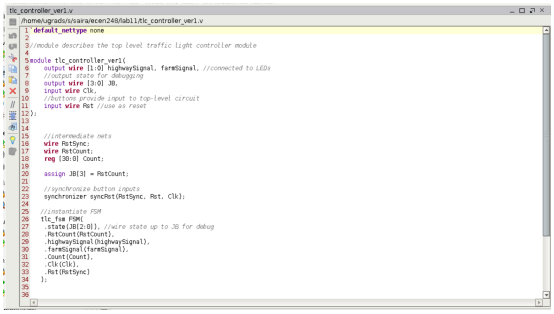
Conclusion:

Finite-state machines were explored further in this lab, but the focus was put on Mealy machines. The ZYBO Z7-10 board was explored further using the buttons and LEDs once again to test the code. Delays were calculated and a state diagram was drawn up in order to understand the machine more.

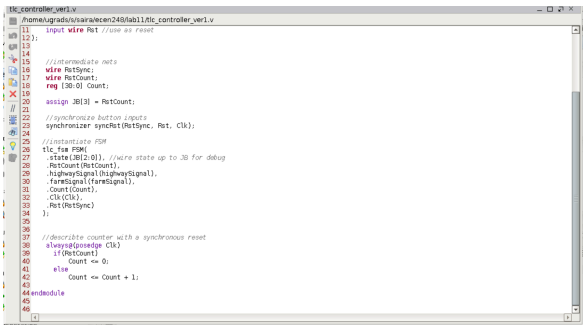
Post-lab Deliverables:



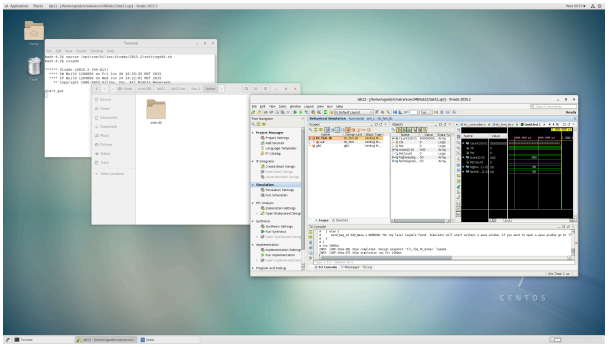
TLC bitstream



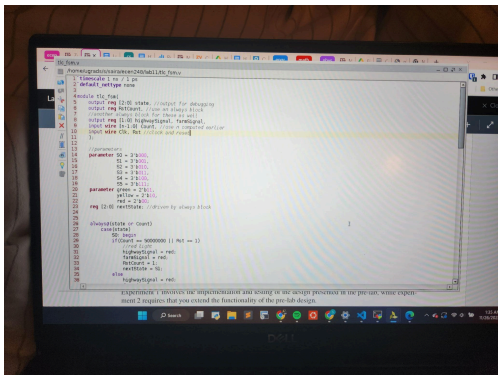
TLC controller code

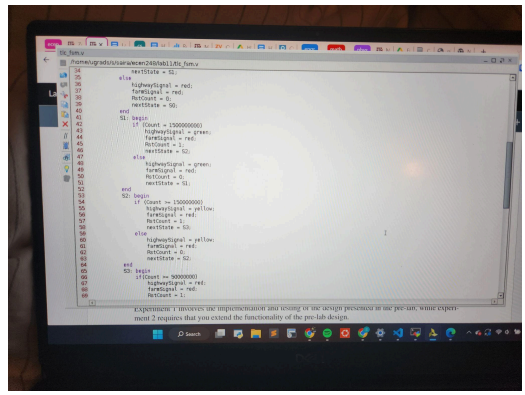


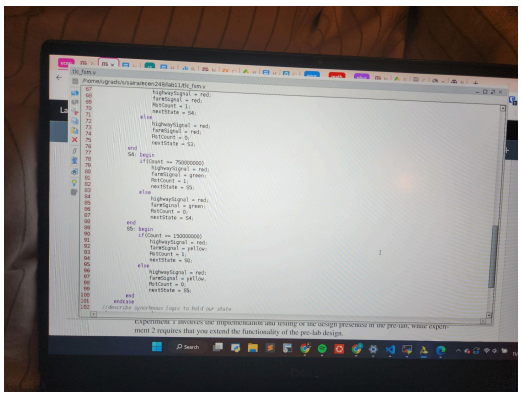
TLC controller Code

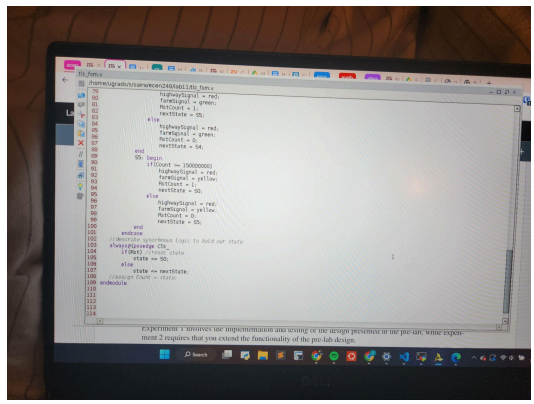


TLC waveform









TLC code

farmsensor.v

/home/ugrads/s/saira/ecen248/lab11/farmsensor.v

```
1 `timescale 1 ns/ 1 ps
2 `default_nettype none
3
4 //light controller module
5
6 module tlc_controller_verl{
7     output wire [1:0] highwaySignal, farmSignal, //connected to LEds
8     output wire [3:0] JB,
9     input wire Clk,
10    input wire Rst,
11    input wire farmSensor
12 };
13
14    //intermediate gates
15    wire RstSync;
16    wire RstCount;
17    wire sensorSync;
18    assign JB[3] = RStCount;
19    //synchronize button inputs
20    synchronizer syncRst(RstSync, Rst, Clk);
21    synchronizer syncSensor(sensorSync, farmSensor, Clk);
22
23    tlc_frsm FSM{
24        .state(JB[2:0]),
25        .RstCount(RstCount),
26        .highwaySignal(highwaySignal),
27        .farmSignal(farmSignal),
28        .Count(Count),
29        .Rst(RstSync),
30        .Clk(Clk),
31        .farmSensor(sensorSync)
32    };
33
34    //describe counter with a syncrhnous reset
35
36    always@(posedge Clk)
37     if(RstCount)
38         Count <= 0;
39     else
40         Count <= Count +1;
41 endmodule
```

code with farmSensor